

Сервисный модуль Interprocess Storage. Локальное хранилище данных

Модуль ядра платформы **Interprocess Storage** и входящая в его состав библиотека **sbis-interprocess-storage** предоставляют реализацию локального хранилища данных, устроенного по принципу ключ-значение, хранящего данные в разделяемой памяти сервера, позволяющего записывать, прочитывать и удалять данные различных типов.

Хранилище реализуется двумя C++ классами - **InterprocessStorage** и **InterprocessStorageHolder**.

Класс **InterprocessStorage** представляет собой "контейнер", хранящий собственно данные.

Класс **InterprocessStorageHolder** представляет собой синглтон, хранящий экземпляры "контейнеров" типа *InterprocessStorage*, и позволяющий создавать их по названию.

Описываемое локальное хранилище данных доступно также и из языка Python, соответствующее API описано ниже, требуется подключить модуль *Interprocess Storage-Py*.

Размер хранилища определяется параметром *Хранилище.МаксимальныйРазмерХранилища*. При достижении указанного размера начинается вытеснение ключей, которые наиболее давно не использовались. Сохранность данных хранилища не гарантируется.

API C++

InterprocessStorageHolder

Данный класс хранит экземпляры объектов типа *InterprocessStorage* и позволяет создавать их, указывая название.

- Статический метод Instance

```
1 InterprocessStorageHolder& InterprocessStorageHolder::Instance();
```

Позволяет получить единственный экземпляр класса **InterprocessStorageHolder**.

- Методы Get

```
1 InterprocessStorage InterprocessStorageHolder::Get( StringView const&
  name );
```

и CreateNew(

```
1 std::unique_ptr< InterprocessStorage >
  InterprocessStorageHolder::CreateNew( StringView const& name );
```

позволяют создавать контейнер — объект типа **InterprocessStorage**, возвращаемый, соответственно, по значению или по указателю. В качестве аргумента указывается название контейнера.

Поскольку контейнеры хранят данные в разделяемой памяти сервера, то эти данные могут быть доступны из разных процессов, выполняющихся на сервере. Соответственно, если в разных процессах создать контейнеры с одинаковыми названиями, то они будут иметь доступ к одним и тем же данным, так что один процесс может записать данные, а другой — прочитывать их. Таким образом может осуществляться обмен данными между процессами в рамках одного сервера.

InterprocessStorage

Данный класс представляет собой "контейнер", хранящий данные. Позволяет читать, записывать и удалять данные.

- Шаблонный метод Read

```
1 template< typename T >
```

```
2 | boost::optional< T > Read( StringView const& key ) const
```

Прочитать из контейнера элемент указанного в шаблоне типа по ключу.

Параметр

key — ключ.

Возвращает элемент по значению.

При отсутствии элемента возвращает **boost::none**.

При ошибке чтения (например, при попытке прочитать данные не того типа, что были записаны) возвращает **boost::none** и пишет предупреждение в лог.

- Шаблонный метод **Write**

Записать в контейнер элемент указанного в шаблоне типа по ключу.

Параметры:

- **key** — ключ.
- **value** — значение.

Возвращает **true**, если значение было записано.

- Шаблонные методы **WriteArray**

```
1 | template< typename T >
2 | size_t WriteArray( std::vector< StringView > const& keys,
   | std::vector< T > const& values )
1 | template< typename KeyIterator, typename ValueIterator >
2 | size_t WriteArray( KeyIterator keys_begin, KeyIterator keys_end,
   | ValueIterator values_begin, ValueIterator values_end )
1 | template< typename T >
2 | size_t WriteArray( std::vector< std::pair< StringView, T > > const&
   | pairs )
1 | template< typename PairIterator >
2 | size_t WriteArray( PairIterator pairs_begin, PairIterator pairs_end )
```

Записать в контейнер множество элементов указанного в шаблоне типа по множеству ключей.

Параметры (в зависимости от варианта перегрузки метода):

- **keys** — ключи,
- **values** — значения,
- **pairs** — пары "ключ + значение",
- **keys_begin, keys_end** — итераторы для ключей (начало, конец),
- **values_begin, values_end** — итераторы для значений (начало, конец),
- **pairs_begin, pairs_end** — итераторы для пар "ключ + значение" (начало, конец).

Возвращает количество записанных элементов.

- Метод **Remove**

```
1 | bool Remove( StringView const& key );
```

Удалить из контейнера элемент по ключу.

Параметр

key — ключ.

Возвращает **true**, если элемент существовал и был удален.

- Метод **RemoveArray**

```
1 | size_t RemoveArray( Vector< StringView >::type const& key );
```

Проверить существование элемента по ключу.

Параметр

key — ключ.

Возвращает **true**, если элемент существует.

- Метод `Clear`

```
1 | bool Clear();
```

Очистить контейнер.

Возвращает **true**, если успешно очищен.

- Метод `TotalMemorySize`

```
1 | size_t TotalMemorySize() const;
```

Получить количество свободных байт памяти в хранилище. Возвращает количество свободных байт памяти.

- Метод `Name`

```
1 | String const& Name() const;
```

Получить название контейнера.

Возвращает название контейнера.

- Шаблонный метод `WriteOrGetExisting`

Записать значение по ключу, если значение для этого ключа отсутствует в контейнере, либо вернуть существующее значение, при наличии в контейнере.

Параметры

- **key** — ключ.
- **value** — значение.

Если значение для переданного ключа `key` отсутствовало в контейнере, и удалось записать переданное значение `value`, то возвращается `boost::none`, иначе возвращается значение, которое есть в контейнере.

API Python

Для использования требуется подключить модуль *Interprocess Storage-Py* и выполнить импорт `import ipc_storage`

Класс **InterprocessStorage** — контейнер, хранящий собственно данные.

- Глобальный метод `GetStorage( name )`

Получить контейнер — объект типа **InterprocessStorage**.

- Методы класса **InterprocessStorage**:

- `ReadString( key )`
- `ReadInt( key )`
- `ReadFloat( key )`
- `ReadBool( key )`
- `ReadTime( key )`
- `ReadDate( key )`
- `ReadDateTime( key )`
- `ReadTimeInterval( key )`
- `ReadHashTable( key )`
- `ReadDecimal( key )`
- `ReadRpcFile( key )`
- `ReadByteVector( key )`
- `ReadRecord( key )`

- ReadRecordSet(key)
- ReadAnyValue(key)

Прочитать элемент по ключу.

Параметр **key** — ключ типа "строка".

Возвращает прочитанный элемент, либо None при отсутствии элемента или ошибке чтения.

- ReadArrayString(keys)
- ReadArrayInt(keys)
- ReadArrayFloat(keys)
- ReadArrayBool(keys)
- ReadArrayDate(keys)
- ReadArrayTime(keys)
- ReadArrayDateTime(keys)
- ReadArrayTimeInterval(keys)
- ReadArrayHashTable(keys)
- ReadArrayDecimal(keys)
- ReadArrayAnyValue(keys)
- ReadArrayRpcFile(keys)
- ReadArrayByteVector(keys)
- ReadArrayRecord(keys)
- ReadArrayRecordSet(keys)

Прочитать список элементов по списку ключей.

Параметр **keys** — список ключей, каждый ключ типа строка.

Возвращает список элементов, среди которых могут быть None в случае отсутствия элемента для соответствующего ключа или ошибки чтения.

- Метод Write

```
1 Write( key, value )
```

Записать элемент по ключу.

Параметры:

- **key** — ключ типа "строка".
- **value** — значение поддерживаемого типа (список поддерживаемых типов соответствует списку методов для чтения элементов).

Возвращает количество успешно записанных элементов: при успешной записи — 1, иначе — 0.

Записать элементы по ключам.

Ключи должны быть типа строка.

Значения должны быть поддерживаемого типа (список поддерживаемых типов соответствует списку методов для чтения элементов).

Возвращает количество успешно записанных элементов.

```
1 Write( { key1: value1, key2: value2, ... } )
2 Write( [ ( key1, value1 ), ( key2, value2 ), ... ] )
```

- Метод Remove

```
1 Remove( key )
```

Удалить элемент по ключу.

Параметр **key** — ключ типа "строка".

Возвращает **True**, если элемент существовал и был успешно удален.

- Метод RemoveArray

```
1 RemoveArray( keys )
```

Удалить из контейнера множество элементов по множеству ключей.

Параметр **keys** — список ключей, каждый ключ типа "строка".

Возвращает количество удаленных элементов.

- Метод HasValue

```
1 HasValue( key )
```

Проверить существование элемента по ключу.

Параметр **key** — ключ, типа строка.

Возвращает **True**, если элемент существует.

- Метод Clear

```
1 Clear()
```

Очистить контейнер.

Возвращает **True** при успешной очистке.